Runtime Governance Architecture: Consistent Governance Execution for Enterprise Systems and Autonomous Agents

John M. Willis
Sustainable Future Tech
jwillis@sustainablefuturetech.com
ORCID: 0009-0007-5889-9628

March 13, 2026

Abstract

Enterprise environments increasingly consist of heterogeneous systems, distributed automation pipelines, and autonomous agents that continuously propose and execute system mutations. To enable consistent governance across such environments, this paper introduces the Runtime Governance Architecture (RGA), a framework that compiles governance intent into canonical artifacts and evaluates them through a deterministic governance execution model. Maintaining consistent governance across these environments is difficult because identity semantics, action representations, policy languages, authorization models, and audit mechanisms differ across platforms.

We first identify the governance functions that must be performed consistently across enterprise stacks and the challenges of implementing them in distributed environments. We then introduce the Artificial Intelligence Governance Control Plane (AGCP) as a reference implementation of the governance execution model defined by RGA. AGCP centralizes governance evaluation into a deterministic pipeline capable of producing consistent authorization decisions and ordered lifecycle evidence.

Within this model, enterprise systems and autonomous agents operate as interacting state machines whose transitions are governed by RGA. This architecture reframes enterprise governance as a runtime transition-control system in which proposed system mutations are evaluated through a deterministic governance pipeline prior to execution.

Keywords: AI governance, runtime governance architecture (RGA), governance control plane, policy compilation, deterministic execution, formal execution semantics, invariant enforcement, autonomous systems, access control, auditability

Cite as: Willis, J. (2026). *Runtime Governance Architecture: Consistent Governance Execution for Enterprise Systems and Autonomous Agents.* Sustainable Future Tech Publishing.
https://doi.org/10.5281/zenodo.19286845

1 Introduction

Enterprise systems increasingly operate as distributed operational environments composed of cloud platforms, infrastructure orchestration systems, application frameworks, and autonomous automa-

tion agents that continuously propose and execute system mutations. While these systems provide powerful automation capabilities, enterprise governance mechanisms remain fragmented across heterogeneous platforms, making it difficult to evaluate governance rules consistently prior to execution. This challenge is amplified as AI-driven systems and autonomous agents increasingly participate in operational decision-making across enterprise infrastructure.

Traditional governance mechanisms rely heavily on documentation, organizational processes, or post-execution audit analysis. However, distributed and automated systems require governance enforcement to occur at runtime, before operational state transitions occur.

Classical security architecture has long recognized the need for trusted enforcement mechanisms that mediate security-relevant operations. The complete mediation principle requires that every security-sensitive operation be evaluated through a trusted enforcement mechanism (Saltzer and Schroeder, 1975). Early protection models similarly emphasized the need for architectural mechanisms that ensure access control decisions are evaluated consistently (Lampson, 1971).

These principles motivate the need for an architectural approach to governance enforcement that ensures operational actions are evaluated consistently at runtime.

This paper proposes a control-plane architecture for governance enforcement in which enterprise systems and autonomous agents submit proposed operational state transitions to a governance execution layer that deterministically evaluates admissibility prior to execution.

This architectural perspective treats governance as a runtime mediation problem rather than a purely procedural or documentation-based process.

Under this model, governance becomes an execution-time control function that determines whether proposed state transitions are admissible before they are applied to operational systems.

2 Related Work

Several areas of prior research are relevant to the governance architecture proposed in this paper, including classical protection models, control-plane architectures in distributed systems, and modern policy-as-code governance frameworks.

2.1 Protection Models and Complete Mediation

Early work in computer security established the principle that security-sensitive operations must be mediated by trusted enforcement mechanisms. Lampson's protection model (Lampson, 1971) and the complete mediation principle described by Saltzer and Schroeder (Saltzer and Schroeder, 1975) emphasize that every access to a protected object must be validated by a trusted mechanism.

The Runtime Governance Architecture applies this principle to operational system mutations. Instead of mediating only data access, governance mediation evaluates proposed operational state transitions prior to execution. This extends classical protection concepts into modern distributed operational environments.

2.2 Control-Plane Architectures

Control-plane separation is a widely used architectural pattern in distributed systems and networking. Clark (Clark, 1988) described the separation of policy decision logic from packet forwarding in early Internet architecture. More recent systems such as Software Defined Networking further

formalize this separation by centralizing control-plane decision logic while data-plane components execute operational actions (McKeown et al., 2008).

The Runtime Governance Architecture applies a similar architectural principle to enterprise governance. Governance logic is implemented within a governance control plane that evaluates proposed operational mutations, while execution systems perform operational actions only after authorization is granted.

2.3 Distributed Authorization Systems

Recent distributed authorization systems have explored mechanisms for evaluating access decisions consistently across large-scale distributed environments. Capability-based authorization approaches such as Macaroons allow decentralized services to enforce contextual authorization constraints through chained authorization tokens (Birgisson et al., 2014). Similarly, large-scale infrastructure systems have implemented centralized authorization services that evaluate access policies across globally distributed systems. For example, the Zanzibar authorization system provides a globally consistent authorization service capable of evaluating access decisions across large distributed systems (Pang et al., 2019). These systems demonstrate the importance of centralized policy evaluation for distributed infrastructure, although they primarily address identity authorization and access control rather than the broader governance of operational state transitions considered in this paper.

2.4 Infrastructure Orchestration and Admission Control

Modern infrastructure orchestration systems increasingly incorporate policy evaluation mechanisms. Large-scale cluster management systems such as Borg, Omega, and Kubernetes implement admission control and scheduling policies that govern workload placement and resource usage (Burns et al., 2016).

However, these mechanisms typically operate within individual platforms rather than providing governance mediation across heterogeneous enterprise systems. The Runtime Governance Architecture generalizes the concept of admission control by defining a governance execution layer capable of evaluating operational mutations originating from multiple enterprise systems and automation agents.

2.5 Policy-as-Code and Declarative Governance

Recent governance frameworks frequently adopt policy-as-code approaches that allow governance constraints to be expressed in machine-evaluable form. Systems such as Open Policy Agent provide declarative policy evaluation engines that can be integrated into application and infrastructure environments (Open Policy Agent, n.d.).

While such systems provide flexible policy evaluation mechanisms, they do not themselves define a governance execution architecture that ensures consistent mediation of operational state transitions across enterprise systems. The Runtime Governance Architecture positions policy evaluation as one component within a broader governance control-plane architecture responsible for deterministic governance execution.

Together these prior works provide important architectural and conceptual foundations for governance enforcement. The Runtime Governance Architecture builds upon these principles by

defining a unified governance execution framework in which operational state transitions across heterogeneous enterprise systems are mediated by a deterministic governance control plane.

3 Contributions

This paper introduces an architectural framework for consistent governance execution across enterprise systems and autonomous agents. The primary contributions are:

1. The definition of a Runtime Governance Architecture (RGA) that provides a canonical representation and execution framework for governance evaluation across enterprise operational environments.

2. The formalization of the core governance execution functions required to evaluate proposed operational state transitions consistently across distributed systems.

3. The design of a governance control-plane architecture, implemented through the Artificial Intelligence Governance Control Plane (AGCP), that centralizes governance evaluation and authorization artifact generation.

4. A governance invocation pipeline that demonstrates how enterprise systems and autonomous agents submit proposed state transitions to a control-plane governance engine for deterministic admissibility evaluation.

4 Runtime Governance Architecture

The Runtime Governance Architecture (RGA) provides the architectural framework within which governance intent is translated into operational enforcement. Governance intent originates from organizational policy processes, compliance frameworks, regulatory requirements, and internal governance boards.

Within the RGA model, governance intent is compiled into canonical artifacts that can be evaluated consistently across enterprise systems. These artifacts define identity and context models, canonical action models, policy and constraint models, authorization artifacts, and lifecycle evidence records.

RGA therefore establishes a governance execution model in which proposed system mutations are evaluated against governance constraints prior to execution.

The Runtime Governance Architecture organizes governance evaluation into a layered execution model in which governance intent is compiled into canonical artifacts that are evaluated prior to operational execution (see Figure 1).

Architecturally, this separation between governance decision logic and operational execution mirrors the control-plane separation used in distributed networking architectures. Control planes coordinate decision logic and policy enforcement across heterogeneous systems, while data-plane systems perform operational execution. Software-defined networking architectures illustrate how centralized policy and routing decisions can be coordinated across distributed infrastructure components (Clark, 1988; McKeown et al., 2008; Kreutz et al., 2015). The Runtime Governance Architecture applies a similar separation to governance enforcement by mediating operational state transitions through a governance control plane.

RGA does not prescribe a particular implementation technology or enforcement mechanism. Instead, it defines an architectural framework for representing governance intent and evaluating governance constraints during system operation. Specific governance control planes, such as AGCP, can implement this execution model while preserving interoperability across heterogeneous enterprise platforms.

5 Governance Execution Functions

To consistently govern system mutations across heterogeneous enterprise platforms, several governance execution functions must be performed.

5.1 Identity and Context Resolution

Governance evaluation requires resolving the identity of actors proposing system actions and the contextual attributes associated with the execution environment. These attributes may include workload identities, device posture signals, environmental context, and tenant-specific configuration.

Because enterprise environments frequently include both human operators and autonomous software agents, identity resolution must accommodate multiple identity types, including user identities, service accounts, workload identities, and machine-generated credentials. Contextual signals may also include runtime attributes such as network location, device security posture, execution environment metadata, and tenant-specific policy context.

5.2 Action Normalization

Systems capable of performing operational actions must represent those actions in a canonical form that can be evaluated consistently. This canonical action representation allows governance rules to be evaluated independently of platform-specific command formats.

Operational systems frequently represent actions using platform-specific interfaces such as API calls, orchestration commands, or infrastructure configuration updates. Action normalization converts these platform-specific representations into a canonical governance object that can be evaluated consistently by the governance control plane.

5.3 Governance Evaluation

Proposed actions must be evaluated against governance rules, policy constraints, and system invariants. Deterministic evaluation of governance rules is required to determine whether the proposed mutation satisfies policy constraints and system invariants.

Modern enterprise systems increasingly implement such governance rules using declarative policy evaluation and policy-as-code mechanisms (Anderson, 2020; Open Policy Agent, n.d.). Distributed authorization systems such as Macaroons and Zanzibar further demonstrate how authorization decisions can be evaluated consistently across large distributed infrastructures (Birgisson et al., 2014; Pang et al., 2019). These mechanisms allow governance constraints to be expressed in a machine-evaluable form and evaluated consistently during system operation.

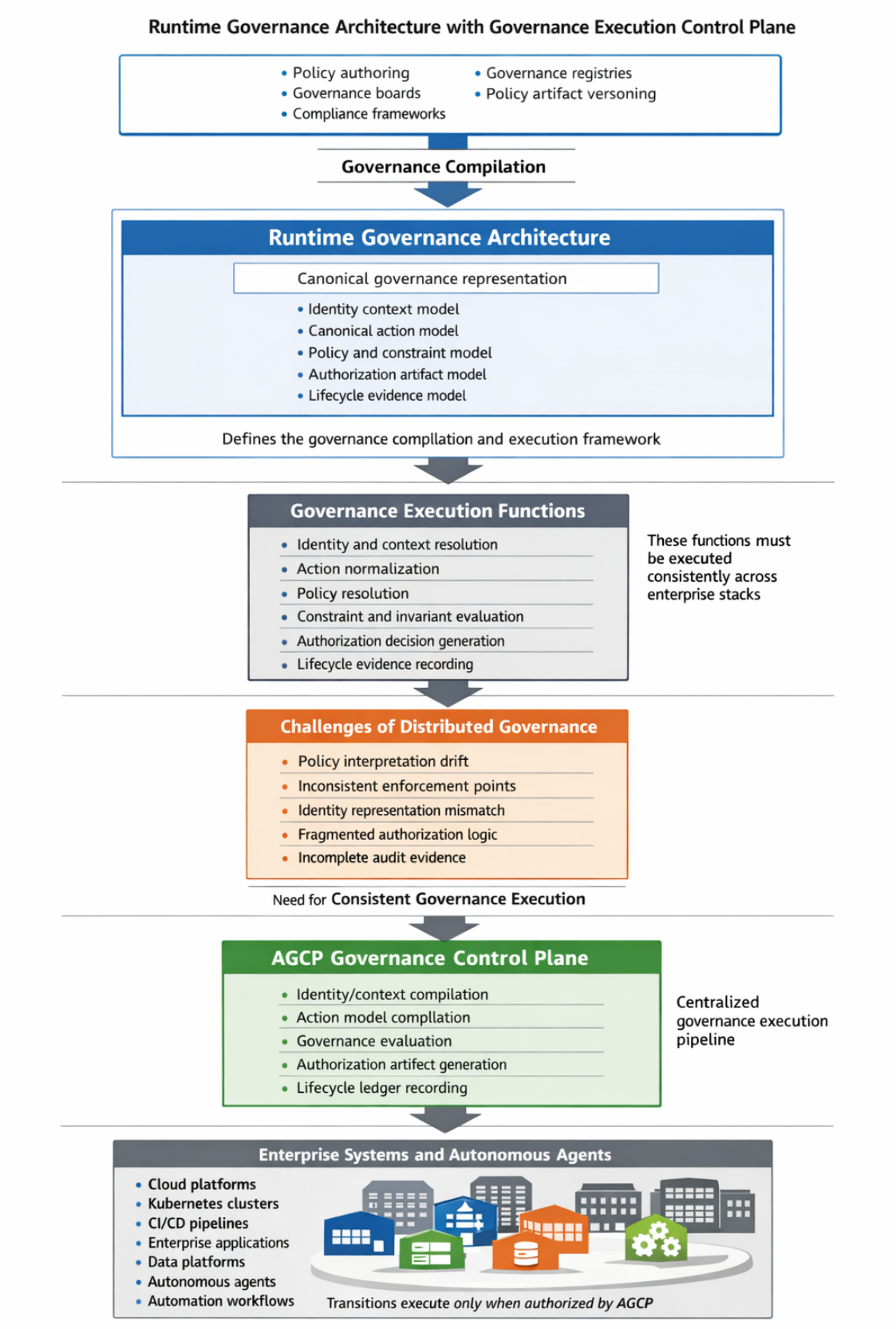

Figure 1: Runtime Governance Architecture

Within the Runtime Governance Architecture, governance evaluation determines whether a proposed system mutation satisfies all applicable governance constraints before the mutation is permitted to execute.

5.4 Authorization Decision Generation

When governance evaluation completes, the system must generate an authorization artifact describing whether the proposed mutation may proceed. This artifact functions as the execution gate that allows or denies the proposed operational transition.

Authorization artifacts may contain references to the evaluated governance rules, the resolved identity context, the normalized action representation, and the resulting authorization decision. These artifacts provide verifiable evidence that the proposed transition satisfied governance constraints at the time of evaluation.

5.5 Lifecycle Evidence Recording

Governance evaluation must produce ordered lifecycle evidence describing the evaluation process and resulting authorization decisions. This evidence allows governance decisions to be reconstructed during later audits or incident investigations.

Lifecycle evidence may include governance evaluation inputs, policy evaluation results, authorization artifacts, and execution outcomes. Recording these events in an append-only lifecycle ledger allows governance state to be reconstructed directly from system history.

In practice, these functions govern operational state transitions generated by enterprise systems and automated agents. For example, a cloud infrastructure change request, an automated deployment pipeline action, or an AI-generated remediation operation may each represent a proposed mutation of system state.

Concrete examples of governed actions include:

- A Kubernetes admission request proposing the deployment of a container workload.
- A CI/CD pipeline proposing the promotion of a build artifact into a production environment.
- A cloud infrastructure controller proposing a modification to network configuration or access policies.
- An autonomous remediation agent proposing a configuration change in response to a detected anomaly.

Each such operation represents a proposed state transition whose admissibility must be evaluated by the governance control plane prior to execution. Within the Runtime Governance Architecture, operational systems and autonomous agents therefore submit proposed mutations as governed actions whose admissibility must be evaluated before execution is permitted.

5.6 Governance Invariants

Within the Runtime Governance Architecture, governance enforcement is guided by a set of invariants that must hold for any operational state transition to be considered admissible. These

invariants define the fundamental conditions under which the governance control plane may authorize execution.

Identity Validity Invariant. Every proposed operational transition must be associated with a resolved identity and execution context. The governance control plane must be able to determine the actor or system responsible for the proposed action and the contextual attributes associated with the execution environment.

Canonical Action Representation Invariant. All proposed operational mutations must be represented in a canonical action model before governance evaluation occurs. Platform-specific command formats must therefore be normalized into governance objects that can be evaluated consistently across heterogeneous enterprise systems.

Policy Evaluation Invariant. Every proposed state transition must be evaluated against all applicable governance rules, constraints, and policy artifacts prior to execution. Operational mutations may not bypass policy evaluation or be executed outside the governance decision pipeline.

Authorization Artifact Invariant. Operational execution may proceed only when the governance control plane generates a valid authorization artifact confirming that governance evaluation has succeeded. The absence of such an artifact implies that the proposed transition is not admissible.

Lifecycle Evidence Invariant. All governance decisions and execution outcomes must be recorded as ordered lifecycle evidence. This evidence allows governance state to be reconstructed from system history and supports audit, compliance verification, and incident investigation.

Together these invariants define the minimal conditions required for deterministic governance enforcement within the Runtime Governance Architecture. The Artificial Intelligence Governance Control Plane implements these invariants during the governance evaluation pipeline described in Section 11.

6 Challenges of Distributed Governance Execution

Although the governance functions described above are conceptually straightforward, implementing them consistently across distributed enterprise systems presents significant challenges.

6.1 Policy Interpretation Drift

Different systems may interpret policy rules differently depending on their implementation language or execution environment.

6.2 Inconsistent Enforcement Points

Some platforms enforce governance rules prior to execution while others rely on monitoring or alerting after execution has already occurred. This behavior contrasts with classical protection architectures that require mediation of security-relevant operations (Lampson, 1971; Saltzer and Schroeder, 1975).

6.3 Identity Representation Mismatch

Enterprise systems frequently use incompatible identity formats, making it difficult to evaluate governance rules consistently.

6.4 Fragmented Authorization Logic

Governance logic implemented independently across multiple platforms can produce inconsistent authorization decisions.

6.5 Incomplete Audit Evidence

Distributed enforcement mechanisms may produce fragmented or incomplete audit trails.

These challenges illustrate the difficulty of implementing consistent governance execution in distributed environments.

7 Threat Model and Trust Assumptions

The Runtime Governance Architecture and the AGCP governance control plane are designed to mediate operational state transitions proposed by heterogeneous enterprise systems and automation agents. Because these systems may operate across distributed environments and administrative domains, governance enforcement must account for both accidental failures and malicious behavior.

This section describes the threat model and trust assumptions under which the architecture is intended to operate.

7.1 Threat Model

The architecture assumes that operational systems and automation agents may propose state transitions that violate governance constraints. Such violations may occur due to configuration errors, software defects, compromised systems, or malicious automation logic.

Potential threats include:

Unauthorized Operational Mutations. A system or agent may attempt to perform a state transition that violates enterprise policy constraints, such as modifying infrastructure configuration, changing access permissions, or deploying unauthorized workloads.

Policy Bypass Attempts. Operational systems may attempt to execute actions without submitting them through the governance control plane, either intentionally or due to misconfiguration of enforcement mechanisms.

Identity Spoofing or Context Manipulation. Attackers may attempt to falsify identity attributes or environmental context information in order to obtain authorization for a prohibited action.

Automation Errors. Autonomous remediation agents or automation pipelines may propose incorrect or unsafe actions in response to incomplete or incorrect signals.

Audit Tampering Attempts. Attackers may attempt to alter or delete records describing governance decisions or execution events in order to conceal unauthorized activity.

The Runtime Governance Architecture mitigates these threats by ensuring that operational state transitions are evaluated by the governance control plane prior to execution and by recording ordered lifecycle evidence describing governance decisions.

7.2 Trust Assumptions

The architecture assumes the presence of several trusted components that collectively enforce governance mediation.

Governance Control Plane Integrity. The governance control plane implementation (e.g., AGCP) is assumed to execute deterministically and to evaluate governance rules correctly. The control plane is treated as a trusted decision authority responsible for generating authorization artifacts.

Policy Artifact Integrity. Governance rules, policy definitions, and invariant specifications are assumed to be compiled through trusted governance processes and stored in tamper-resistant governance registries.

Enforcement Point Integrity. Operational execution environments are assumed to enforce authorization decisions produced by the governance control plane. Policy enforcement points such as admission controllers, API gateways, or orchestration systems must prevent execution when authorization artifacts are absent or invalid.

Ledger Integrity. The lifecycle evidence ledger used to record governance decisions is assumed to provide append-only guarantees sufficient to prevent unauthorized modification of historical governance records.

Under these assumptions, the governance control plane acts as the authoritative mechanism that determines whether proposed operational state transitions are admissible. Systems and automation agents may propose mutations, but execution occurs only when the governance control plane authorizes the transition.

8 Governance Control Planes

One architectural approach to addressing these challenges is to centralize governance evaluation within a governance control plane responsible for evaluating proposed system mutations before they are executed.

In this model, operational systems propose actions to the governance control plane, which evaluates those actions against governance constraints and returns an authorization decision.

This separation of governance evaluation from operational execution follows the architectural pattern of control-plane and data-plane separation widely used in distributed systems (Clark, 1988; McKeown et al., 2008).

9 The AGCP Governance Control Plane

The Artificial Intelligence Governance Control Plane (AGCP) provides a reference implementation of the governance execution model defined by the Runtime Governance Architecture (Willis, 2026).

AGCP implements the governance execution control plane required to evaluate and authorize operational actions within the Runtime Governance Architecture. The AGCP specification defines

the canonical submission model, evaluation pipeline, authorization artifact generation, and append-only lifecycle ledger used to enforce governance decisions across enterprise systems (Willis, 2026).

Within the RGA model, AGCP performs the runtime governance evaluation required to determine whether a proposed operational state transition may proceed. Governance rules compiled within the RGA framework are evaluated by AGCP against the identity context, action representation, and applicable policy artifacts associated with a proposed action.

The governance control plane architecture and its relationship to enterprise execution environments are illustrated in Figure 2.

AGCP centralizes governance evaluation into a deterministic execution pipeline responsible for the governance execution functions defined by the AGCP specification (Willis, 2026):

- identity and context compilation
- canonical action modeling
- governance rule evaluation
- authorization artifact generation
- lifecycle ledger recording

By centralizing these governance execution functions, AGCP enables consistent governance evaluation across heterogeneous enterprise systems.

10 Enterprise Systems and Autonomous Agents

Within the Runtime Governance Architecture, enterprise systems and autonomous agents operate as operational state machines that propose system mutations.

These systems may include:

- cloud platforms
- container orchestration systems
- CI/CD pipelines
- enterprise applications
- data processing platforms
- autonomous automation agents
- AI-driven decision systems
- machine-learning inference services

Many modern orchestration platforms implement partial governance enforcement mechanisms through admission control and policy engines (Burns et al., 2016).

However, RGA positions governance evaluation as a separate architectural concern implemented through the governance control plane.

In this model, enterprise systems and autonomous agents propose operational state transitions that are evaluated by the governance control plane prior to execution.

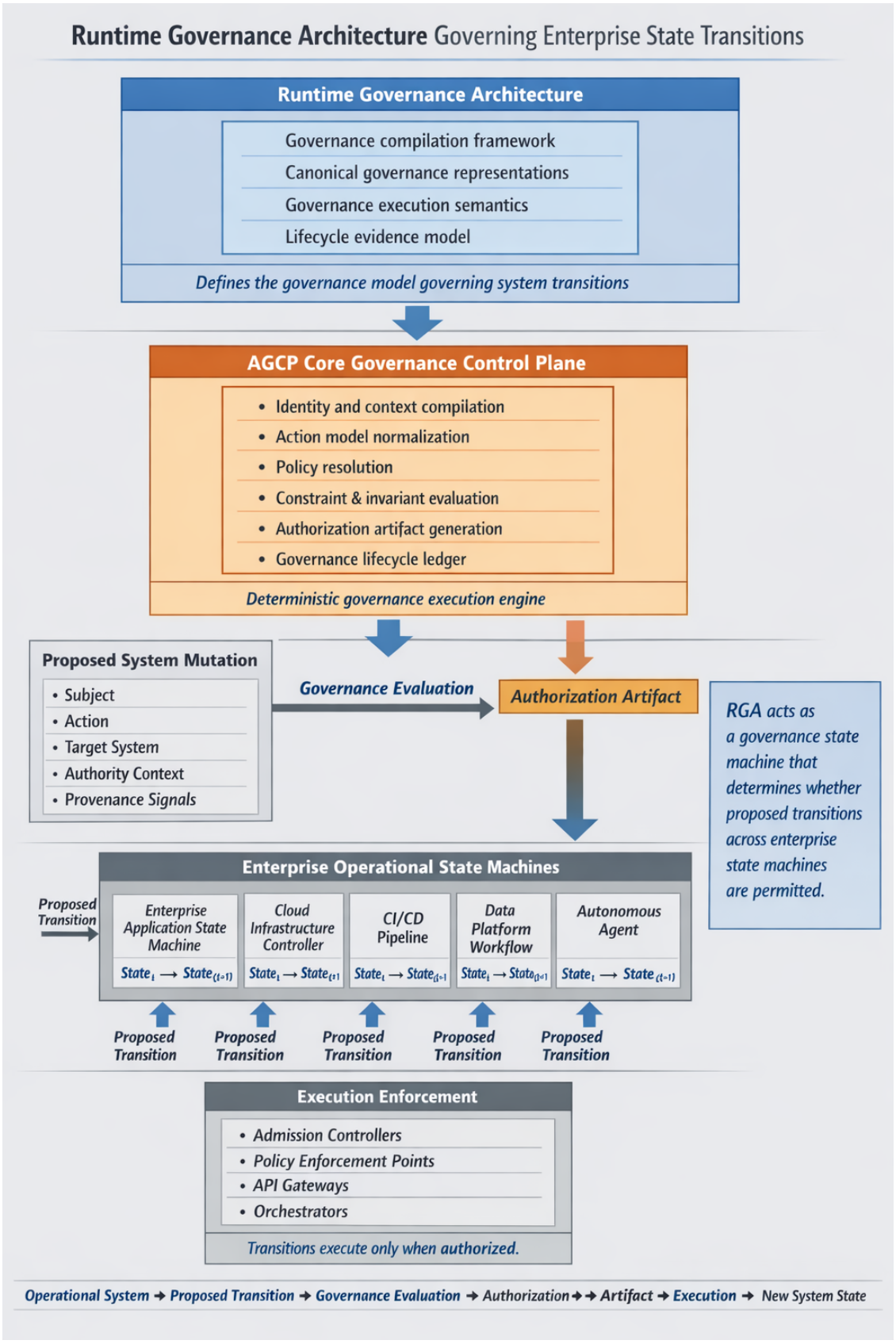

Figure 2: AGCP Governance Control Plane

11 AGCP Invocation Across the Governance Pipeline

Within the Runtime Governance Architecture, the governance control plane is invoked at specific decision boundaries of the governance execution pipeline.

Operational systems and autonomous agents do not directly execute state transitions. Instead, proposed transitions are submitted to the governance control plane for evaluation and authorization. At each stage of this pipeline the governance control plane acts as the deterministic decision authority that evaluates whether the proposed transition may proceed. Operational systems therefore interact with AGCP at defined decision boundaries rather than executing mutations directly.

The governance pipeline can be summarized as five stages.

Stage 1 collects identity, tenant, provenance, and request context information from external systems.

Stage 2 constructs the canonical representation of the proposed action and normalizes the operational request into a governance object.

Stage 3 performs governance rule evaluation. Policies, constraints, and invariants defined within the governance framework are evaluated to determine whether the proposed transition satisfies all required conditions.

Stage 4 generates the authorization artifact that gates execution. If governance evaluation succeeds, the control plane produces an authorization reference that allows the action to proceed. If governance evaluation fails, the rejection and associated reason are recorded.

Stage 5 records lifecycle outcomes and execution evidence. After execution occurs, the governance control plane records the execution result and updates the lifecycle state associated with the action.

This invocation model ensures that governance evaluation occurs at each decision boundary of the control-plane pipeline. Operational state transitions are permitted only when governance evaluation produces a valid authorization artifact. This structure parallels control-plane mediation models used in distributed systems, where operational mutations are executed only after policy evaluation authorizes the transition. Similar patterns appear in distributed authorization systems that centralize policy evaluation across heterogeneous services, such as Zanzibar-style authorization architectures (Pang et al., 2019).

Within the Runtime Governance Architecture, the boundary between governance evaluation and authorization artifact generation functions as the deterministic commit point at which governance authority is resolved prior to execution.

12 Evaluation and Application Discussion

To illustrate the applicability of the Runtime Governance Architecture and the AGCP governance control plane, this section briefly examines how the architecture applies across several classes of enterprise systems. These examples are not intended as empirical evaluations but rather as architectural case analyses demonstrating how governance mediation can be applied consistently across heterogeneous operational environments.

12.1 Infrastructure Orchestration

Modern infrastructure platforms such as Kubernetes clusters, infrastructure-as-code frameworks, and cloud orchestration systems continuously generate operational mutations. Examples include container deployments, network configuration changes, and infrastructure scaling operations.

Many orchestration platforms implement partial governance enforcement mechanisms through admission control and policy engines (Burns et al., 2016).

Within the Runtime Governance Architecture, such operations are treated as proposed state transitions submitted to the governance control plane. The canonical action representation allows orchestration events to be evaluated consistently against governance rules before execution.

For example, a Kubernetes admission request proposing a workload deployment may be normalized into a governance action object and evaluated against policy constraints such as workload identity restrictions, resource isolation policies, or security configuration requirements. Execution proceeds only when the governance control plane produces an authorization artifact confirming that the proposed mutation satisfies governance constraints.

12.2 Financial Transaction Systems

Financial systems routinely perform high-value state transitions such as payment authorization, account balance updates, and transaction settlement operations. These transitions require strict governance and auditability.

Within the Runtime Governance Architecture, financial transaction requests can be represented as governed operational mutations. Governance evaluation can verify identity context, transaction constraints, regulatory compliance conditions, and fraud detection signals prior to transaction execution.

The lifecycle evidence ledger produced by the governance control plane provides a verifiable record of authorization decisions and execution outcomes, supporting audit reconstruction and regulatory compliance.

12.3 Security Automation

Security automation platforms increasingly deploy automated remediation actions in response to detected threats. Examples include blocking network traffic, rotating credentials, quarantining workloads, or modifying access policies.

Because automated remediation actions may themselves introduce operational risk, governance mediation is necessary to ensure that automated responses satisfy organizational constraints.

Within the Runtime Governance Architecture, security automation systems operate as proposers of remediation actions. These actions are evaluated by the governance control plane prior to execution, ensuring that automated responses satisfy policy constraints and system invariants.

12.4 Industrial Control Systems

Industrial control systems manage physical infrastructure such as energy grids, manufacturing systems, and transportation networks. Operational commands in these environments can have direct physical consequences.

Applying governance mediation in such environments allows operational commands to be evaluated prior to execution against safety constraints, authorization rules, and operational policies. For example, a command to modify a turbine configuration or alter manufacturing parameters could be evaluated by the governance control plane before execution.

Although industrial control environments often require low-latency decision paths, the governance pipeline can still provide deterministic admissibility evaluation at operational decision boundaries.

These examples illustrate how the Runtime Governance Architecture generalizes across enterprise environments in which operational systems generate state transitions that must be governed prior to execution.

13 Conclusion

Enterprise governance programs define extensive policy and compliance requirements intended to regulate operational behavior across complex technical environments. However, governance intent defined through policy documents does not automatically translate into consistent enforcement within operational systems.

The Runtime Governance Architecture describes how governance intent can be compiled into canonical governance artifacts that enable deterministic governance evaluation at runtime. By separating governance compilation from governance execution, the RGA framework identifies the functions required to evaluate governance decisions consistently across distributed enterprise platforms.

The Artificial Intelligence Governance Control Plane provides the governance execution infrastructure required to implement this model. AGCP acts as the runtime enforcement layer that evaluates proposed operational state transitions, determines governance admissibility, and produces authorization artifacts that gate execution. Governance decisions are recorded through append-only lifecycle evidence, allowing governance state to be derived directly from system history.

Within this architecture, enterprise systems and autonomous agents operate as proposers of operational state transitions rather than direct executors. Proposed transitions are submitted to the governance control plane, evaluated against governance rules and invariants, and authorized only when governance conditions are satisfied.

By treating governance enforcement as a control-plane execution problem, this architecture provides a deterministic mechanism for ensuring that operational behavior remains consistent with enterprise governance intent across heterogeneous platforms and distributed operational environments.

Copyright

Code and Specification Availability

The Artificial Intelligence Governance Control Plane (AGCP) specification and related artifacts referenced in this paper are publicly available.

AGCP Specification (archived release):
https://doi.org/10.5281/zenodo.18923302

AGCP Reference Repository:
https://github.com/jwillisSFT/agcp-spec

Patent Notice

Elements of the architecture described in this work may be covered by pending patent applications. Publication of this paper does not grant a license to any patent rights associated with the described systems or methods. The author reserves all patent rights associated with the described systems and methods in any pending patent applications.

References

Anderson, R. (2020). *Security Engineering: A Guide to Building Dependable Distributed Systems.* 3rd Edition. Wiley. https://www.cl.cam.ac.uk/~rja14/book.html

Birgisson, A., Politz, J., Van den Hooff, J., McAuley, D., Erlingsson, Ú., Morrisett, G., and Smith, J. (2014). Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. *Proceedings of the Network and Distributed System Security Symposium (NDSS 2014).* Available at: https://www.ndss-symposium.org/ndss2014/ndss-2014-programme/macaroons-cookies-contextual-caveats-decentralized-authorization-cloud/

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5), 50–57. https://doi.org/10.1145/2890784

Clark, D. (1988). The Design Philosophy of the DARPA Internet Protocols. *ACM SIGCOMM Computer Communication Review*, 18(4). https://doi.org/10.1145/52324.52336

Kreutz, D., Ramos, F., Veríssimo, P., Rothenberg, C., Azodolmolky, S., and Uhlig, S. (2015). Software-defined networking: A comprehensive survey. *Proceedings of the IEEE*, 103(1), 14–76. https://doi.org/10.1109/JPROC.2014.2371999

Lampson, B. W. (1971). Protection. *Proceedings of the Fifth Princeton Symposium on Information Sciences and Systems.* Reprinted in *ACM SIGOPS Operating Systems Review*, 8(1), 1974. Available at: https://bwlampson.site/08-Protection/WebPage.html

McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., and Turner, J. (2008). OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2), 69–74. https://doi.org/10.1145/1355734.1355746

Open Policy Agent. Open Policy Agent Documentation. https://www.openpolicyagent.org

Saltzer, J., and Schroeder, M. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. https://doi.org/10.1109/PROC.1975.9939

Pang, R., Caceres, R., Burrows, M., Chen, Z., Dave, P., Germer, N., Golynski, A., Graney, K., Kang, N., Kissner, L., Korn, J. L., Parmar, A., Richards, C. D., and Wang, M. (2019). Zanzibar: Google's Consistent, Global Authorization System. *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19).* Renton, WA. Available at: https://research.google/pubs/pub48190/

Willis, J. M. (2026). *Artificial Intelligence Governance Control Plane (AGCP) Specification v0.9.0.* Sustainable Future Tech Publishing. https://doi.org/10.5281/zenodo.18923302